Kara Muhafız — Gün 4-5 Çalışma Raporu

Manuel Sürüş Node'u, Klavye Teleop ve Araç Yönelim Düzeltmesi (2026-10-04)

---

1. Hedef

Gün 3'te araç Ackermann controller ile doğrudan topic komutuyla hareket ettirilebiliyordu. Bugünün hedefi: bunu gerçek bir MANUEL SÜRÜŞ SİSTEMİNE dönüştürmek — standart teleop girdisini güvenlik limitleriyle controller'a aktaran bir node, ve klavyeyle rahat sürülebilen bir kontrol deneyimi.

2. Yapılan Değişiklikler

Yeni paket: karamuhafiz_control (ament_python — ilk kez gerçek Python kodu yazdığımız paket)

- package.xml, setup.py, setup.cfg, resource/ işaret dosyası — ament_python paket iskeleti
- manual_drive_node.py — /cmd_vel → güvenlik limitleri + işaret düzeltme + komut-kesilirse-dur (watchdog) → /ackermann_steering_controller/reference_unstamped
- keyboard_teleop_node.py — terminal odaklıyken çalışan ok tuşlu teleop (kalıcı gaz + anlık direksiyon tasarımı)
- global_keyboard_teleop_node.py — pynput ile PENCERE ODAĞINDAN BAĞIMSIZ çalışan, gerçek tuşa-bas/bırak olaylarıyla tam simültane çoklu tuş destekli teleop

karamuhafiz_bringup paketinde değişenler

- config/manual_drive.yaml — manual_drive_node parametreleri (hız/dönüş limitleri, command_timeout, invert_linear)
- launch/manual_drive.launch.py — sim_bringup.launch.py'yi dahil edip manual_drive_node'u da başlatan üst launch dosyası
- launch/sim_bringup.launch.py — spawn_yaw launch argümanı eklendi (aracın başlangıç yönelimini kaynak koda gömmeden ayarlanabilir yaptı)
- package.xml — karamuhafiz_control ve teleop_twist_keyboard bağımlılıkları eklendi

karamuhafiz.urdf.xacro, teknofest_parkur.world ve mesh dosyalarının içeriği yine hiç değiştirilmedi — yönelim düzeltmesi dahil tüm ayarlar bizim launch/config dosyalarımızda yapıldı.

3. Ulaşılan Sonuç

- Araç artık doğru yönde (burnu parkurda ilerlenecek yöne bakacak şekilde) spawn oluyor.
- İleri ok = araç gerçekten ileri (burun yönünde) gidiyor; sol/sağ ok = beklenen yönde dönüyor.
- İki farklı teleop seçeneği çalışır durumda: terminal-tabanlı ve pencere odağından bağımsız global klavye kontrolü.
- 6 sonar sensörünün tümünün gerçek mesafe verisi yayınladığı doğrulandı (ros2 topic echo ile), yerleşimlerinin (4 yana + 2 arkaya bakan, 360° çevre koruması) doğru/tasarlandığı gibi olduğu teyit edildi.

4. Karşılaşılan Sorunlar ve Çözümleri

4.1 teleop_twist_keyboard tek seferlik yayın yapıyor

Belirti: Tuşa basınca araç kısa hareket edip duruyordu, sürekli basmak gerekiyordu. Sebep: teleop_twist_keyboard yalnızca her tuş vuruşunda TEK mesaj yayınlıyor; bizim manual_drive_node'daki command_timeout (0.3s) güvenlik ayarı, tuşlar arası doğal boşlukta aracı durduruyordu. İlk çözüm (geçici): aracın repeat_rate parametresiyle çalıştırılması önerildi, ama sonrasında kendi klavye node'umuzu yazmaya karar verildi (daha esnek kontrol için).

4.2 Aynı anda iki tuşa basınca hareket duruyordu

Sebep: X11/terminal klavye tekrarı AYNI ANDA yalnızca TEK tuş için üretilir — ikinci bir tuşa basılınca işletim sistemi ilk tuşun tekrarını durdurup ikinciye geçer (donanım/OS sınırı, kod hatası değil). Çözüm (terminal node'unda): gaz (ileri/geri) KALICI yapıldı — yalnızca Asağı ok veya BOŞLUK değiştirir, başka tuşa geçilince kaybolmaz; direksiyon (sol/sağ) ise bırakınca kendiliğinden düzleniyor. Kalıcı/daha iyi çözüm: pynput kütüphanesiyle GERÇEK tuşa-bas/bırak olaylarını yakalayan global_keyboard_teleop_node yazıldı — bu, gerçek simültane çoklu tuş desteği sağlıyor (terminal sınırlamasını tamamen aşıyor) ve ekstra olarak Gazebo penceresi odaktayken de çalışıyor.

4.3 Önemli keşif: Araç parkura ters (kuyruğu ileride) spawn oluyordu

Belirti: Kullanıcı Gazebo'da gözlemledi — ön tekerlekler/ön kamera, parkurda ilerlenecek yönün TERSİNE bakıyordu. Önemli yeniden değerlendirme: Gün 3'teki "ackermann_steering_controller'ın iç yön kuralı ters" teşhisi muhtemelen YANLIŞTI — o günkü gözlem (pozitif linear.x aracı geri sürüyordu) aslında aracın baştan yanlış yönde doğmasının bir SONUCU olabilir: araç kendi burnu yönünde (doğru/beklenen şekilde) hareket ediyordu, sadece burnu zaten ters yöne bakıyordu. Çözüm: sim_bringup.launch.py'ye spawn_yaw launch argümanı eklendi (spawn_entity.py'nin -Y bayrağıyla, varsayılan 180°); manual_drive.yaml'daki invert_linear false yapıldı. Test edildi: araç artık doğru yönde doğuyor VE ileri komut doğru yönde çalışıyor — hem yönelim hem işaret sorunu TEK kökten (yanlış spawn açısı) çözülmüş oldu.

Not: Bu, xacro/world dosyalarına dokunmadan yapıldı — spawn açısı yalnızca bizim launch dosyamızdaki bir parametre, verilen dosyaların içeriği hâlâ tamamen orijinal.

4.4 Klavye kontrolü yalnızca kendi terminali odaktayken çalışıyordu

Sebep: keyboard_teleop_node, ham terminal girdisini (stdin) okuyor — bu yalnızca o terminal penceresi aktif/odaktayken çalışır. Çözüm: pynput kütüphanesi (pip3 install --user pynput) ile X11 seviyesinde GLOBAL klavye dinleme yapan global_keyboard_teleop_node yazıldı — Gazebo penceresi aktifken de çalışıyor. Önce oturumun X11 mi Wayland mı olduğu kontrol edildi (pynput'un global kanca atması yalnızca X11'de güvenilir çalışır); kullanıcının oturumu X11 çıktı, sorunsuz ilerlendi.

5. Kod Analizi — Yeni Dosyaların İncelenmesi

5.1 manual_drive_node.py — komut adaptörü/watchdog çekirdeği

```
def _tick(self) -> None:
    elapsed_s = (self.get_clock().now() - self._last_cmd_time).nanoseconds / 1e9
    out = Twist()
    if elapsed_s > self._command_timeout:
        out.linear.x = 0.0; out.angular.z = 0.0   # guvenlik: dur
    else:
        linear = _clamp(self._last_cmd.linear.x, self._max_linear)
        angular = _clamp(self._last_cmd.angular.z, self._max_angular)
        out.linear.x = -linear if self._invert_linear else linear
        out.angular.z = angular
    self._pub.publish(out)
```

Bu fonksiyon bir ROS 2 Timer ile sabit aralıklarla (20 Hz) çağrılır — /cmd_vel'den yeni mesaj gelsin gelmesin HER zaman bir şey yayınlar. elapsed_s hesaplaması, son komuttan bu yana geçen süreyi ROS saatiyle (get_clock().now(), simülasyon zamanına duyarlı) ölçer. _clamp fonksiyonu hız/dönüşü güvenli sınırların dışına çıkarmaz — bu, şartnamenin "araç sınırlarını aşmamalı" kuralının doğrudan kod karşılığı.

5.2 setup.py — entry_points ile çalıştırılabilir tanımlama

```
entry_points={
    'console_scripts': [
        'manual_drive_node = karamuhafiz_control.manual_drive_node:main',
        ...
    ],
},
```

Bu satır, colcon'a "manual_drive_node adında bir çalıştırılabilir oluştur, çalıştırıldığında karamuhafiz_control paketindeki manual_drive_node.py dosyasının main() fonksiyonunu çağır" der.

Bu sayede ros2 run karamuhafiz_control manual_drive_node komutu çalışır hale gelir — ament_cmake paketlerindeki CMakeLists.txt'nin install() mantığının Python karşılığı.

5.3 sim_bringup.launch.py — spawn_yaw launch argümanı

```
spawn_yaw_arg = DeclareLaunchArgument(
    "spawn_yaw", default_value="3.14159265",
    description="Aracin baslangic yonelimi (yaw, radyan)."
)
spawn_yaw = LaunchConfiguration("spawn_yaw")
...
arguments=["-file", sdf_path, "-entity", "karamuhafiz",
           "-z", "0.05", "-Y", spawn_yaw]
```

DeclareLaunchArgument, bu launch dosyasının komut satırından değiştirilebilir bir "girdi" tanımlamasını sağlar (ör. ros2 launch ... spawn_yaw:=0.0) — parametrenin koda gömülmemesi gerektiği için (şartname kuralı) sabit bir sayı yazmak yerine bu yöntem seçildi. LaunchConfiguration, bu değere launch dosyasının başka yerlerinden erişmeyi sağlayan bir "değişken referansı"dır. -Y bayrağı, spawn_entity.py'nin desteklediği başlangıç yönelimi (yaw, radyan) parametresidir.

5.4 manual_drive.launch.py — launch dosyalarını birleştirmek

```
sim_bringup = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(bringup_share, "launch", "sim_bringup.launch.py")
    )
)
```

IncludeLaunchDescription, bir launch dosyasının İÇİNE başka bir launch dosyasını "çağırmasını" sağlar — kod tekrarı yapmadan (sim_bringup'ı kopyalamadan) onun üzerine manual_drive_node'u ekleyebildik. Bu desen ileride autonomous_drive.launch.py yazarken de aynı şekilde kullanılacak — her ikisi de aynı temel simülasyonu paylaşacak.

6. Bugün Çalışılması Gereken Kavramlar

- ament_python paket yapısı: setup.py, setup.cfg, entry_points, resource/ işaret dosyası — ament_cmake'ten farkı
- ROS 2 node sınıfı yazmak: rclpy.node.Node miras alma, create_subscription/create_publisher/create_timer
- ROS 2 parametre deklare etme (declare_parameter/get_parameter) ve bunun YAML dosyalarıyla ilişkisi
- Watchdog/timeout deseni: "son veriden bu yana ne kadar süre geçti" mantığının güvenlik kritik sistemlerde neden yaygın olduğu
- Launch argümanları: DeclareLaunchArgument + LaunchConfiguration ile parametrik launch dosyaları yazmak
- IncludeLaunchDescription ile launch dosyalarını modüler birleştirmek
- X11 klavye giriş sınırlamaları ve global input hook (pynput/XRecord) kavramı — terminal (TTY) girdisi ile gerçek tuşa-bas/bırak olayları arasındaki fark
- Robotun yönelimi (yaw) ve spawn-time pose parametreleri (-x/-y/-z/-Y)

7. Sıradaki Adım (Gün 6-7)

Ayrı bir acil duruş (e-stop) mekanizması (anında, komut akışından bağımsız tam durdurma), parkurun tamamında manuel tur, ve manuel sürüş rosbag kaydı. Ardından Hafta 2'ye (sensörler + lokalizasyon) geçilecek.

Bu rapor, Kara Muhafız ROS 2 2025 Parkur Görevi 4 haftalık yol haritasının Hafta 1 / Gün 4-5 kapsamını özetler. Önceki günler: Gun1-2_Calisma_Raporu.pdf, Gun3_Calisma_Raporu.pdf. Genel kavramlar için: Konu_Anlatimi_ROS2.pdf.